

Оглавление

Реферат	2
Введение	3
1 Анализ рекрутмента, методов NLP и архитектуры семантического ранжирования	6
1.1 Системный анализ предметной области рекрутмента, процессов найма и ATS-систем	6
1.2 Аналитический обзор методов обработки естественного языка (NLP) и алгоритмов машинного обучения (Learning to Rank)	7
1.3 Обоснование выбора архитектуры нейросетевых моделей (BERT, Sentence-BERT) и методов векторизации	9
2 Проектирование данных, архитектуры обработки и интеграции с ATS	11
2.1 Выполнение сбора, профилирования, очистки и разметки исходных данных (вакансии и резюме)	11
2.2 Проектирование архитектуры пайплайна предварительной обработки и модуля семантического анализа	12
2.3 Спецификация программного интерфейса (API) и модулей интеграции с ядром ATS	13
3 Разработка и реализация модуля семантического ранжирования кандидатов	19
3.1 Разработка программного кода подготовки данных и ML-пайплайна	19
3.2 Реализация и дообучение (fine-tuning) модели сопоставления «вакансия-резюме»	19
4 Экспериментальная оценка, интеграция и анализ эффективности модуля ранжирования кандидатов	28
4.1 Проведение экспериментальной оценки качества алгоритма ранжирования по метрикам NDCG, Precision@K и др.	28
4.2 Выполнение интеграции созданного модуля в целевую ATS-систему	29
4.3 Оценка экономической и практической эффективности внедрения в бизнес-процесс	31
Заключение	34
Источники	35

Реферат

Пояснительная записка содержит ____ страниц, 2 рисунка, 5 таблиц. Количество использованных источников – 25. Количество приложений – 0.

Разработан прототип модуля семантического ранжирования кандидатов для ATS-системы. Реализованы контур подготовки данных, дообучение bi-encoder модели на парах «вакансия–резюме», модуль ранжирования, FastAPI-сервис и интеграция с тестовым контуром ATS Reqcore.

Ключевые слова: ATS, рекрутмент, резюме, вакансия, семантическое ранжирование, bi-encoder, Sentence-BERT, cosine similarity, FastAPI, Reqcore.

Целью данной работы является разработка и экспериментальная проверка прототипа модуля семантического сопоставления пары «вакансия–резюме» для интеграции в ATS-систему.

В первой главе анализируются предметная область рекрутмента, процессы найма, ATS-системы, методы обработки естественного языка и подходы к ранжированию кандидатов. Рассматриваются классические методы информационного поиска, Learning to Rank, архитектуры BERT и Sentence-BERT, а также ограничения, связанные со справедливостью алгоритмического ранжирования.

Во второй главе проектируется контур обработки данных и интеграции с ATS. Описываются сбор, очистка и разметка данных вакансий и резюме, архитектура пайплайна предварительной обработки, модуль семантического анализа, API-интерфейс и связь bi-encoder сервиса с ядром ATS Reqcore.

В третьей главе описывается практическая реализация модуля семантического ранжирования. Приводятся сведения о подготовке обучающих пар, дообучении модели cointegrated/rubert-tiny2, структуре сохраненного артефакта, гиперпараметрах обучения, реализации класса ATSRanker и FastAPI-сервиса для ранжирования кандидатов.

В четвертой главе проводится экспериментальная оценка качества разработанного модуля. Рассматриваются метрики бинарного сопоставления и ranking-метрики, результаты сравнения базовой и дообученной модели, проверка API-интеграции с Reqcore, а также расчет практической эффективности применения bi-encoder подхода для предварительного отбора кандидатов.

В приложениях дополнительные материалы не приводятся.

Введение

Современный рекрутмент характеризуется высокой нагрузкой на этапе первичного отбора кандидатов. В корпоративной практике на одну вакансию может поступать значительное число откликов, поэтому ручной анализ всех резюме становится трудозатратным и плохо масштабируемым процессом. В этих условиях особую роль приобретают системы управления кандидатами — Applicant Tracking Systems, ATS, которые позволяют хранить данные о вакансиях и кандидатах, управлять стадиями воронки найма, фиксировать результаты рассмотрения откликов и поддерживать принятие решений рекрутером [2].

Одной из ключевых задач ATS является ранжирование кандидатов относительно конкретной вакансии. В такой постановке описание вакансии или набор требований к позиции можно рассматривать как запрос, а резюме кандидатов — как документы, которые необходимо упорядочить по степени релевантности [2, 3]. При этом задача отличается от классического информационного поиска: результат ранжирования влияет не только на удобство работы пользователя, но и на видимость кандидатов в процессе найма. Кандидаты, оказавшиеся в верхней части списка, получают больше внимания со стороны рекрутера, тогда как релевантные профили на нижних позициях могут быть фактически не рассмотрены. Поэтому для систем автоматизированного отбора важны не только точность ранжирования и производительность, но и воспроизводимость оценки, интерпретируемость результата и контроль ограничений применяемого алгоритма [2, 16].

Актуальность работы обусловлена необходимостью разработки программных модулей, способных выполнять семантическое сопоставление вакансий и резюме на основе современных методов обработки естественного языка. Традиционные подходы, основанные на ключевых словах и простом совпадении терминов, не всегда позволяют корректно учитывать смысловую близость текстов, различия в формулировках требований и описаний опыта, а также неполноту данных в резюме. Использование нейросетевых моделей представления текста, в частности BERT и Sentence-BERT, позволяет перейти от лексического сравнения к сравнению плотных векторных представлений, что делает возможным более гибкое ранжирование кандидатов [5, 6]. Вместе с тем применение таких моделей в контуре ATS требует не только выбора архитектуры, но и подготовки данных,

построения воспроизводимого пайплайна обучения, реализации API-интеграции и экспериментальной проверки качества.

Целью работы является разработка и экспериментальная проверка прототипа модуля семантического сопоставления пары «вакансия–резюме» для интеграции в ATS-систему.

Для достижения поставленной цели в работе решаются следующие задачи:

1. Выполнить анализ предметной области рекрутмента, процессов найма, ATS-систем и существующих подходов к ранжированию кандидатов.
2. Рассмотреть методы обработки естественного языка и обучения ранжированию, применимые к задаче сопоставления вакансий и резюме.
3. Обосновать выбор bi-encoder архитектуры на базе Sentence-BERT для построения семантических представлений текстов.
4. Спроектировать контур подготовки данных, включающий очистку, деидентификацию, формирование текстовых представлений и разметку пар «вакансия–резюме» на основе исторических HR-событий.
5. Реализовать программный модуль ранжирования кандидатов и API-сервис для вычисления релевантности пары «вакансия–резюме».
6. Выполнить дообучение модели на подготовленном наборе данных и провести экспериментальную оценку качества по метрикам бинарного сопоставления и ranking-метрикам.
7. Проверить возможность интеграции разработанного модуля с тестовым контуром ATS Reqscore и оценить практическую эффективность использования bi-encoder подхода для предварительного отбора кандидатов.

Объектом исследования являются процессы автоматизированного сопоставления вакансий и резюме в ATS-системах.

Предметом исследования являются методы семантического кодирования текстов вакансий и резюме, алгоритмы вычисления меры релевантности и программная архитектура интеграции модуля ранжирования в ATS.

Научная новизна работы заключается в адаптации bi-encoder подхода к задаче сопоставления русскоязычных вакансий и резюме на основе исторических HR-событий, а также в экспериментальной проверке качества такого подхода на сформированном корпусе пар «вакансия–резюме». В отличие от простого поиска по ключевым словам, предлагаемый подход использует семантические векторные представления текстов и позволяет оценивать

близость вакансии и резюме с учетом смыслового содержания, а не только прямого совпадения терминов.

Теоретическая значимость работы состоит в систематизации подходов к семантическому ранжированию кандидатов, рассмотрении применимости методов Learning to Rank, BERT и Sentence-BERT к задаче рекрутмента, а также в определении ограничений bi-encoder архитектуры при использовании в ATS-сценариях. В работе отдельно фиксируются границы применимости разработанного решения: текущий прототип предназначен для числового семантического скоринга и предварительного отбора кандидатов, но не заменяет полноценную экспертную или LLM-интерпретацию причин соответствия кандидата вакансии.

Практическая значимость работы заключается в разработке прототипа ML-модуля, который может использоваться в ATS для предварительного ранжирования кандидатов. В рамках работы реализованы подготовка данных, дообучение модели cointegrated/rubert-tiny2, модуль ранжирования, FastAPI-сервис и интеграция с тестовым контуром ATS Reqcore. Такой модуль может применяться как первый этап отбора: он позволяет быстро получить числовую оценку соответствия кандидата вакансии и сократить число профилей, передаваемых на более дорогую экспертную или LLM-оценку.

Методологическую основу работы составляют методы обработки естественного языка, нейросетевые модели контекстных представлений, подходы к обучению ранжированию, методы оценки качества ранжирования, а также практики проектирования программных API и ML-пайплайнов. В качестве базовой архитектуры используется bi-encoder модель, формирующая независимые эмбединги вакансии и резюме с последующим вычислением косинусной меры сходства.

Работа состоит из введения, четырех разделов, заключения и списка использованных источников. В первом разделе рассматриваются предметная область рекрутмента, ATS-системы, методы NLP и архитектуры семантического ранжирования. Во втором разделе описываются проектирование данных, архитектура обработки и интеграция с ATS. В третьем разделе приводится описание реализации модуля семантического ранжирования, подготовки обучающих данных, дообучения модели и API-сервиса. В четвертом разделе выполняется экспериментальная оценка качества модели, рассматривается интеграция с Reqcore и анализируется практическая эффективность применения bi-encoder подхода. В заключении формулируются основные результаты работы, ограничения текущей реализации и направления дальнейшего развития.

1 Анализ рекрутмента, методов NLP и архитектуры семантического ранжирования

1.1 Системный анализ предметной области рекрутмента, процессов найма и ATS-систем

Являясь программным ядром современного рекрутинга, системы класса ATS поддерживают как минимум два критически важных и взаимосвязанных процесса: управление воронкой найма (от формирования лонг-листа к шорт-листу) и поддержку принятия решений посредством ранжирования и фильтрации больших пулов кандидатов [2].

С точки зрения системного анализа, рекрутмент можно представить как потоковую систему обработки заявок. Входом системы являются профили и резюме кандидатов, обогащенные текстовыми и структурированными атрибутами, а выходом — решения о продвижении кандидата по стадиям воронки и формирование на ранних этапах упорядоченного списка для изучения рекрутером. Особенность данной предметной области заключается в том, что ранжирование кандидатов представляет собой высокорисковое управленческое решение (high-stakes decision): в отличие от ранжирования электронных документов, соискатели обладают собственными правами и интересами, а алгоритмическое ранжирование выступает инструментом поддержки принятия решений человеком, а не автономной системой выбора [2].

Практический эффект алгоритмов ранжирования на популяцию кандидатов объясняется преимущественно моделью просмотра списков: позиции, находящиеся в топе выдачи, получают непропорционально большую долю внимания (экспозицию). В контексте управления персоналом данный эффект способен многократно усиливать различия между группами соискателей даже при незначительных отличиях в исходных оценках релевантности [2]. Таким образом, справедливость подобных систем зависит не только от лежащего в их основе алгоритма, но и от интерфейса самой ATS, а также от стратегии просмотра списков пользователем [2].

Исторически инструменты, ориентированные на интеграцию с ATS и автоматизацию процессов отбора, чаще всего использовали методы извлечения структурированных атрибутов (навыков, стажа, образования), механизмы фасетного поиска и техники ранжирования из арсенала классического информационного поиска. Характерным примером подобного подхода может служить система PROSPECT, позиционируемая как система поддержки принятия решений. Данный комплекс извлекает ключевые аспекты профиля кандидата, представляет их в виде фасетов и применяет алгоритмы информационного поиска для ранжирования соискателей относительно

требований вакансии. В экспериментальной части исследования авторы отмечают, что использование выявленной структурированной информации повысило качество ранжирования на 30% [3].

Современная эволюция систем рекрутмента характеризуется переходом к семантическому сопоставлению кандидатов на базе плотных векторных представлений (эмбеддингов) и к обучению ранжирующих моделей на основе поведенческой обратной связи (кликов, конверсий на следующие этапы, фактов найма). Современные индустриальные решения используют конвейеры многозадачного обучения, а также алгоритмы на базе больших языковых моделей (LLM) для более глубокого анализа требований вакансии. В частности, передовые архитектуры включают трехступенчатый процесс поиска талантов (отбор кандидатов, предварительное ранжирование, окончательное переранжирование), применяют LLM для извлечения тонких предпочтений рекрутера из текстовых описаний позиций и истории закрытия вакансий, а также задействуют модели класса *mixture-of-experts* с учетом специфики конкретной роли [17].

В контексте настоящего исследования целевая ATS рассматривается как программная среда, для работы внутри которой разрабатываемый модуль должен выполнять следующие функции: 1) принимать на вход текстовое описание вакансии и пул кандидатов; 2) вычислять меру семантической близости и осуществлять эффективное ранжирование; 3) обеспечивать объяснимость результатов выдачи (минимально — через сопоставление извлеченных навыков и атрибутов); 4) предоставлять верифицируемую оценку качества работы [2, 23].

1.2 Аналитический обзор методов обработки естественного языка (NLP) и алгоритмов машинного обучения (Learning to Rank)

Для решения задач сопоставления текстов вакансий и резюме традиционно применяются различные семейства методов представления и сравнения текстовой информации: лексические модели (такие как TF-IDF), вероятностные ранжирующие функции (в частности BM25), а в последнее время — нейросетевые семантические вложения на базе трансформеров. В классической вероятностной парадигме функция BM25 зарекомендовала себя как надежная базовая метрика информационного поиска. Связь показателя релевантности с вероятностными предпосылками, параметрами насыщения частоты термина (*tf*) и нормализацией по длине документа критически важна для текстов вакансий и резюме, поскольку они зачастую существенно различаются по объему и стилистике [11]. В современных системах поиска информации и рекомендаций (в том числе построенных на методологии Learning to Rank) метрика BM25 по-прежнему служит мощным и устойчивым ориентиром; при интеграции нейросетевых алгоритмов

ранжирования в контур ATS требуется наличие воспроизводимой классической базовой линии для проведения сравнительного анализа [8, 11].

Методология обучения ранжированию (Learning to Rank, LTR) формализует процесс построения оптимальной ранжирующей функции по заранее размеченным данным (уровням релевантности или картам предпочтений). Общепринято классифицировать алгоритмы LTR на три основные группы: поточечный подход (pointwise, сводящийся к регрессии или классификации по отдельным объектам), попарный подход (pairwise, обучающийся предсказывать предпочтения между двумя документами) и списочный подход (listwise, оптимизирующий функцию потерь непосредственно на всем списке документов на основе ранжирующих метрик) [8]. Классическим примером попарного подхода выступает архитектура RankNet, в рамках которой задается вероятностная модель предпочтения между парой объектов исходя из разности их индивидуальных оценок, после чего минимизируется кросс-энтропийная ошибка функции потерь логистического связывания. Главное практическое преимущество подобных методов заключается в их дифференцируемости и встроенной совместимости с нейросетевыми архитектурами [9].

Ключевым аспектом LTR является оценка качества получаемого ранжирования. Такие метрики, как дисконтированная кумулятивная выгода (DCG) и нормализованная дисконтированная кумулятивная выгода (NDCG), не только учитывают градуированный уровень релевантности, но и применяют прогрессивный дисконт в зависимости от занятой позиции, что полностью согласуется с когнитивным эффектом позиционного смещения при просмотре списков пользователем [10]. В прикладной сфере рекрутмента метрики семейства NDCG признаются наиболее подходящими, так как HR-специалисты редко просматривают весь перечень соискателей целиком: релевантность первых позиций ранжирования имеет прямое влияние на итоговый результат набора [2, 16].

Самостоятельным фундаментальным направлением исследований алгоритмов машинного обучения выступает обеспечение справедливости ранжирования (fairness in ranking). В подобных формулировках основной акцент делается на распределении экспозиции как ограниченного ресурса. Применяются ограничения на долю экспозиции в зависимости от объективных достоинств кандидата, с оптимизацией метрики полезности (например, алгоритмы policy-gradient могут оптимизировать NDCG при условии выполнения заданных критериев справедливости) [16]. Для архитектуры ATS это представляет значительную концептуальную ценность: вероятность найма кандидата напрямую зависит от занимаемой позиции, и даже незначительные перекосы в базовых скорингах способны резко изменить экспозицию [2, 16].

1.3 Обоснование выбора архитектуры нейросетевых моделей (BERT, Sentence-BERT) и методов векторизации

Современным стандартом анализа семантики текста в области обработки естественного языка выступает архитектура трансформеров, механизм внимания которой позволяет эффективно моделировать смысловые зависимости внутри последовательностей токенов без использования классических сверточных или рекуррентных подходов [7]. На основе этой парадигмы архитектура BERT (Bidirectional Encoder Representations from Transformers) предложила инновационную схему двунаправленного предобучения языковых представлений на больших объемах неразмеченных текстов с их последующим дообучением (fine-tuning) под прикладные задачи [5].

При решении задач семантического сопоставления пар «вакансия–резюме» важнейшим архитектурным решением является выбор между перекрестным энкодером (cross-encoder) и архитектурой на основе би-энкодера (bi-encoder). Перекрестный энкодер производит совместную контекстную обработку пары текстов сквозь всю сеть, требуя высоких вычислительных затрат на формирование предсказания. Би-энкодер генерирует независимые семантические вложения для каждого текста, после чего сравнивает их с помощью стандартной косинусной меры сходства или скалярного произведения. В основополагающей статье по Sentence-BERT (SBERT) подробно раскрыто ограничение перекрестных энкодеров: при использовании классического BERT попарное вычисление мер близости внутри большой базы приводит к непомерно высокому числу циклов инференса. Модели семейства SBERT модифицируют BERT в сиамскую или триплетную архитектуру для генерации оптимизированных вложений предложений, сравнимых напрямую. Это кардинально снижает вычислительные издержки вывода с сохранением высокого качества на бенчмарках семантического сходства текста [6].

В сценарии разработки для ATS вышеупомянутые выводы делают рациональной реализацию архитектуры двухступенчатого ранжирования. Первая ступень — стадия выборки (recall), основанная на поиске по плотным вложениям через би-энкодер для быстрого отбора топ-N кандидатов из глобальной базы. Вторая ступень — стадия переранжирования (re-rank), которая может осуществляться более ресурсоемкой моделью перекрестного энкодера либо строиться как отдельная LTR-надстройка над широким пулом признаков. Подобная логика построения конвейера явно задокументирована в передовых индустриальных исследованиях корпоративного поиска соискателей [17].

Эмпирическую применимость решения на базе SBERT к предметной области найма доказывает статья по архитектуре CareerBERT: авторам удалось перевести резюме соискателей и описания вакансий в единое векторное пространство. Применялась сиамская

нейросеть с функцией потерь попарного ранжирования (Multiple Negatives Ranking loss), формирующая пространство, в котором семантически связанные тексты предельно близки друг к другу, а нерелевантные отдалены [1].

При работе с русскоязычными текстами дополнительными факторами выступают качество готовых весовых моделей для целевого языка, а также доступность референсных метрик бенчмарков. В современных публикациях вводится мультязычный формат оценки Massive Text Embedding Benchmark для русского языка (ruMTEB), предлагаются базовые векторные модели (например, cointegrated/rubert-tiny2) и формализуются категории тестовых задач переранжирования, что имеет критическое значение для тестирования будущего модуля обработки отечественного контента [19, 20].

Выбор методов векторизации обязательно должен учитывать риски алгоритмического смещения: аудиты в системах рекрутмента подтверждают, что использование только векторных эмбедингов может провоцировать дискриминационные искажения результатов (по полу, расе, национальности и объему текстового профиля кандидата) [22].

2 Проектирование данных, архитектуры обработки и интеграции с ATS

2.1 Выполнение сбора, профилирования, очистки и разметки исходных данных (вакансии и резюме)

Сбор текстовой информации в ATS концептуально подразделяется на работу с двумя классами сущностей. С одной стороны, обрабатываются данные вакансий (должностные обязанности, формальные требования, стек технологий, условия труда). С другой стороны, обрабатываются резюме (развернутый опыт работы, ключевые навыки, наличие сертификатов, данные об образовании, стаж работы). Практика систематического извлечения данных аспектов для целей ранжирования была ранее описана как надежная основа первичной обработки соискателей [3].

Профилирование корпуса вакансий и резюме должно протоколировать распределение длины текста, долю шумовой или неформативной информации (маркетинговые блоки, универсальные фразы), плотность доменных терминов, процентную долю смысловых дубликатов, а также языковой состав. В контексте извлечения навыков из резюме существенной проблемой признаются границы определений терминологии: научные обзоры демонстрируют тенденцию к разнообразию в подходах к классификации и стандартизации доменных параметров HR, что обуславливает необходимость создания строгих внутренних регламентов и унификации метрик для разметки данных [18].

Процедура разметки массива данных в рамках проекта протекает на нескольких уровнях:

1. Разметка глобальной релевантности пар «вакансия–резюме» для последующего обучения ранжирующих моделей и контроля качества по NDCG.
2. Детализированная разметка терминов на уровне выделения текстовых фрагментов (span-level) с типизацией скиллов (hard, soft, knowledge) для построения интерпретируемой выдачи.
3. Косвенная разметка посредством накопления поведенческих сигналов рекрутеров (отказы, приглашения на собеседование), реализуемая в условиях промышленной эксплуатации алгоритма.

Ключевым примером при решении задачи разметки является ресурс SKILLSPAN, включающий более 14 тысяч предложений из описаний вакансий, экспертно размеченных согласно строгим гайдлайнам. В исследовании показано, что доменно адаптированные модели машинного обучения (single-task подход) в этой задаче значительно превосходят общие алгоритмы генерации текстов [24]. Основываясь на этом, необходимо: а) актуализировать собственную таксономию навыков; б) выполнять настройку

трансформеров на локальных корпусах ATS; в) оформлять модуль извлечения навыков как независимый подпроцесс [18, 24].

По аналогии с работой CareerBERT [1], при наличии внутреннего резерва данных целесообразно использовать подходы лемматизации и стандартизации профессий по государственным классификаторам или международным таксономиям (например, ESCO). Это снижает терминологический разрыв между кандидатами и рекрутерами [1, 18, 24].

Ввиду наличия конфиденциальных персональных данных в резюме процедуры очистки должны включать деидентификацию. Доказано, что избыточные «сигнальные» поля (индикация локации, ФИО, университета) способны стать фактором предвзятости для трансформерных моделей [22]. Корректная подготовка данных обязана скрывать параметры, не отражающие напрямую профессиональные качества [2, 22]. В реализованном конвейере маскирование персональных данных выполнено методом жесткой подстановки токенов: все обнаруженные паттерны телефонных номеров и адресов электронной почты заменяются соответственно на токены `'[MASKED_PHONE]'` и `'[MASKED_EMAIL]'`, удаляются HTML-теги и шумовые спецсимволы, заполняются пропущенные значения полей. Весь конвейер подготовки данных реализован по принципу Zero-Dependency Batch Processing — на нативном Python с потоковым чтением файлов, что позволяет обрабатывать корпуса объемом в десятки гигабайт без загрузки полного объема информации в оперативную память и повышает воспроизводимость на маломощных серверных конфигурациях.

2.2 Проектирование архитектуры пайплайна предварительной обработки и модуля семантического анализа

Архитектура пайплайна проектируется в виде логически упорядоченной последовательности модулей предпроцессинга:

1. Модуль приема (ingestion): получение профилей кандидатов и описаний позиций из БД комплекса ATS, проверка структурного формата и нормализация кодировок.

2. Модуль лингвистической нормализации: реализация токенизации строк, удаление пунктуации и приведение текста к нижнему регистру (lowercasing). Модуль выявляет язык источника, производит дедупликацию и удаляет юридические штампы.

3. Структурно-аналитический модуль: отвечает за вычленение смысловых кластеров (опыт работы, образование, навыки), экстракцию специализированных терминов по категориям hard/soft skills. Формируется детальная цифровая матрица компетенций профиля [24].

4. Модуль построения эмбеддингов: компонент конвертации нормализованных текстов в плотные векторные представления на базе сиамских архитектур Sentence-BERT.

При необходимости базовые веса моделей обогащаются в ходе доменного дообучения ин-хаус [1, 6, 24].

5. Блок первичной выборки (Recall/Retrieval): отбор узкой первичной выборки лучших кандидатов по критериям семантической схожести векторов. В планах развития проекта осуществление подсчёта схожести векторов через механизмы поиска ближайших приближенных соседей (ANN). Для этого в инфраструктуре закладывается система класса Vector DB (например, Milvus), обеспечивающая гибкую фильтрацию [12].

6. Далее в планах развития проекта идёт модуль алгоритмического переранжирования (Re-ranking): отвечает за финальный пересчет позиций топ-N выборки при помощи LTR-алгоритмов. Учитываются и интегрируются исходные семантические скоры, выявленные структурные пересечения по профессиональному стажу и прокси-метрики поведения [17].

7. Модуль объяснимости (Explainability Engine): формирует интерпретируемые сводки (например, текстовые плашки совпадения навыков). Механизм прозрачности работы модели призван компенсировать фактор эффекта «черного ящика», повышая доверие конечного пользователя внутри ATS к результату алгоритма [23].

8. И наконец в планах развития проекта, чтобы завершающим шагом должен идти модуль LLM-генерации критериев оценки: в целевой ATS-системе (Reqcore) реализован автоматизированный конвейер формирования объективной шкалы оценивания кандидатов на основе текста описания вакансии. Система передает описание позиции в LLM (с поддержкой нескольких провайдеров: OpenAI, Anthropic, Google AI) и получает структурированный ответ, жестко валидируемый JSON-схемой. Каждый сгенерированный критерий содержит уникальный идентификатор, наименование, инструкцию по поиску фактов в резюме, категорию (технический навык, soft skill, образование), базовую шкалу оценки и рекомендованный вес значимости. Полученные критерии далее выступают в роли формализованного чек-листа для автоматического AI-скоринга входящих резюме.

2.3 Спецификация программного интерфейса (API) и модулей интеграции с ядром ATS

API-модуль реализован в файле ``experiments/scripts/l1_api_server.py`` как микросервис FastAPI ``ATS Semantic Ranking Engine API`` версии ``1.0.0``. В текущей реализации сервер использует Pydantic-схемы ``CandidatePayload`` и ``SearchPayload``, временный in-memory индекс ``CANDIDATE_INDEX`` и объект ранжирования ``ATSRanker``, загружаемый при старте приложения. Кандидатский профиль передается в API в уже подготовленном текстовом формате ``model_input``; отдельные публичные маршруты для самостоятельного построения эмбедингов вакансии или резюме в коде не реализованы.

Таблица 1 – API схема микросервиса для ранжирования резюме

Маршрут	Метод	Входная схема	Реализованное поведение	Ответ
<code>/index/candidate`</code>	POST	<code>`CandidatePayload`: `id: str`, `model_input: str`, `metadata: Optional[Dict[str, Any]]`</code>	Формирует словарь кандидата, добавляет обязательные поля <code>`id`</code> и <code>`model_input`</code> , при наличии объединяет их с <code>`metadata`</code> , затем помещает запись в <code>`CANDIDATE_INDEX`</code> .	JSON вида <code>`{"status": "indexed", "total_pool_size": N}`</code> .
<code>/search/rank`</code>	POST	<code>`SearchPayload`: `vacancy_text: str`, `filters: Optional[Dict[str, str]]`, `top_k: int = 10`</code>	Проверяет наличие загруженного <code>`RANKER_ENGINE`</code> , вызывает <code>`ATSRanker.search(vacancy_text, candidate_pool, filters, top_k)`</code> по текущему <code>`CANDIDATE_INDEX`</code> и возвращает результат ранжирования как JSON.	При незагруженной модели возвращает HTTP 500 с <code>`{"error": "Model not loaded"}`</code> ; при штатном выполнении возвращает JSON, сформированный модулем ранжирования.
<code>/score`</code>	POST	<code>`ScorePayload`: `vacancy_text: str`,</code>	Кодирует одну пару «вакансия-резюме» через	JSON вида <code>`{"status": "ok", "match_score": float, "latency_ms": int}`</code>

		<code>`candidate_text: str`</code>	текущий <code>`RANKER_ENGINE.model`</code> , вычисляет косинусное сходство и возвращает одиночный <code>`match_score`</code> . Используется как service-to-service endpoint для ветки <code>`provider`</code> <code>====</code> <code>biencoder`</code> в Reqcore.	или HTTP 500 с <code>`{"error": "Model not loaded"}`</code> .
--	--	------------------------------------	--	---

Таким образом, в текущем коде реализованы три маршрута: ``POST /index/candidate`` для добавления профиля кандидата во временный индекс, ``POST /search/rank`` для ранжирования текущего пула кандидатов относительно текста вакансии и ``POST /score`` как внутренний интеграционный endpoint для Reqcore. Маршруты ``POST /rank``, ``POST /embed/job``, ``POST /embed/resume``, ``POST /index/upsert``, ``POST /index/delete`` и ``POST /monitoring/log`` в текущем коде отсутствуют; это направление будущего расширения API, но не реализованная функциональность.

В текущей версии API реализован базовый набор маршрутов, обеспечивающий индексацию кандидатских профилей, ранжирование текущего пула кандидатов по тексту вакансии и вычисление одиночного score для пары «вакансия–резюме» в сервисной интеграции с Reqcore. В связи с этим дальнейшее развитие API целесообразно связывать не с повторным описанием уже существующей функциональности, а с расширением интерфейса в сторону более полного разделения онлайн- и офлайн-контуров обработки данных, а также с усилением эксплуатационных характеристик системы.

Одним из таких направлений может стать введение отдельных публичных маршрутов для работы с эмбедингами вакансий и резюме. В отличие от текущей реализации, где кандидатский профиль поступает в систему уже в подготовленном текстовом виде, выделение специализированных endpoints для получения векторных представлений позволило бы формализовать этап предрасчёта признаков, упростить

инкрементальное обновление представлений и сделать API более пригодным для сценариев, в которых контур подготовки данных отделён от контура ранжирования. Такое расширение особенно актуально при переходе от прототипной схемы взаимодействия к более устойчивой сервисной архитектуре.

Перспективным направлением также является развитие механизмов синхронизации индекса. В текущем состоянии используется временный in-memory индекс, достаточный для демонстрации и проверки базовой логики поиска, однако в дальнейшем API может быть дополнено операциями обновления и удаления записей, ориентированными на согласованную синхронизацию между ATS и внешним хранилищем представлений кандидатов. Реализация такого слоя позволила бы перейти от эпизодической загрузки данных к управляемому жизненному циклу профилей в системе и тем самым приблизить решение к требованиям промышленной эксплуатации.

Отдельного развития требует и уровень прикладной наблюдаемости. В настоящей версии API отсутствует специализированный маршрут для журналирования результатов работы модели, а потому сбор сведений о качестве выдачи, динамике ранжирования и устойчивости поведения системы остаётся ограниченным. В дальнейшем целесообразно предусмотреть средства централизованной регистрации агрегированных метрик и служебных событий, что создаст основу для систематического мониторинга, сравнительного тестирования версий модели и более прозрачной интерпретации результатов в эксплуатационной среде.

Наконец, дальнейшее расширение возможно на уровне самого поискового маршрута. Если в текущей реализации выдача ориентирована прежде всего на возврат результата ранжирования, то в перспективе API может быть дополнено поддержкой более богатого выходного формата, включающего дополнительные метаданные и интерпретационные компоненты. Подобное развитие позволило бы сделать систему удобнее для практического использования в ATS-сценариях, где важна не только численная релевантность кандидата, но и возможность объяснить основания, по которым профиль оказался выше или ниже в итоговом списке.

Текущая серверная интеграция между ATS Reqcore и модулем ML-оценки реализована в обработчике ``ats/reqcore/server/api/applications/[id]/analyze.post.ts``. Эта точка является фактическим входом для анализа одной заявки и отвечает за выбор ветки скоринга по значению ``ai_config.provider``. В обоих вариантах результат интеграции сводится к обновлению числового поля ``application.score`` и записи аудита в ``analysis_run``; различается только способ получения этого числа.

Основной путь Reqcore в текущей реализации является LLM-ориентированным. Обработчик ``analyze.post.ts`` загружает заявку, AI-конфигурацию организации, критерии оценки из ``scoring_criterion``, текст резюме из ``document.parsedContent``, затем вызывает ``scoreApplication()`` из ``ats/reqcore/server/utils/ai/scoring.ts``. Функция ``scoreApplication()`` формирует структурированный prompt по вакансии, критериям и материалам кандидата и передает его в ``generateStructuredOutput()`` из ``ats/reqcore/server/utils/ai/provider.ts``, где выбирается конкретный провайдер (``openai``, ``anthropic``, ``google`` или ``openai_compatible``). Возвращенный структурированный ответ содержит массив ``evaluations`` с полями ``criterionKey``, ``applicantScore``, ``confidence``, ``evidence``, ``strengths`` и ``gaps``. После этого в ``analyze.post.ts`` вычисляется итоговый балл через ``computeCompositeScore()``, записи по критериям сохраняются в таблицу ``criterion_score``, а агрегированный результат записывается в ``application.score``.

Альтернативный путь интеграции реализован как ветка ``provider === biencoder`` в том же файле ``analyze.post.ts``. В этой ветке Reqcore не вызывает LLM-провайдер и не использует ``scoreApplication()``. Вместо этого обработчик обращается к локальной функции ``scoreWithBiEncoder()``, которая читает ``BIENCODER_URL`` из окружения и отправляет HTTP POST на Python-микросервис ``experiments/scripts/11_api_server.py`` по маршруту ``/score`` с телом ``{ vacancy_text, candidate_text }``. Python-сервис загружает ``ATSRanker`` и напрямую вычисляет cosine similarity между эмбеддингом вакансии и эмбеддингом одного резюме, после чего возвращает ``match_score``. Затем ``analyze.post.ts`` линейно отображает этот score из диапазона ``[-1, 1]`` в шкалу ``0..100`` и записывает результат в ``application.score``. Для этой ветки массив ``evaluations`` остается пустым, а записи в ``criterion_score`` не создаются, поскольку bi-encoder путь в текущем коде предназначен для числового pre-ranking или альтернативного coarse scoring, а не для замены полного критериального LLM-анализа.

Следовательно, фактическая точка интеграции находится не в UI и не в клиентском коде, а в серверном маршруте ``ats/reqcore/server/api/applications/[id]/analyze.post.ts``, где выбирается один из двух путей: ``analyze.post.ts -> scoreApplication() -> provider -> criterion_score -> application.score`` для критериального LLM-scoring и ``analyze.post.ts -> biencoder branch -> HTTP /score -> numeric score -> application.score`` для bi-encoder оценки. С точки зрения архитектуры bi-encoder путь следует описывать как альтернативный или предварительный путь ранжирования, пригодный для дешевого числового отбора, но не как полную замену всей LLM-логики Reqcore, поскольку в нем отсутствуют criterion-level explanations, evidence, strengths/gaps и генерация структурированной интерпретации.

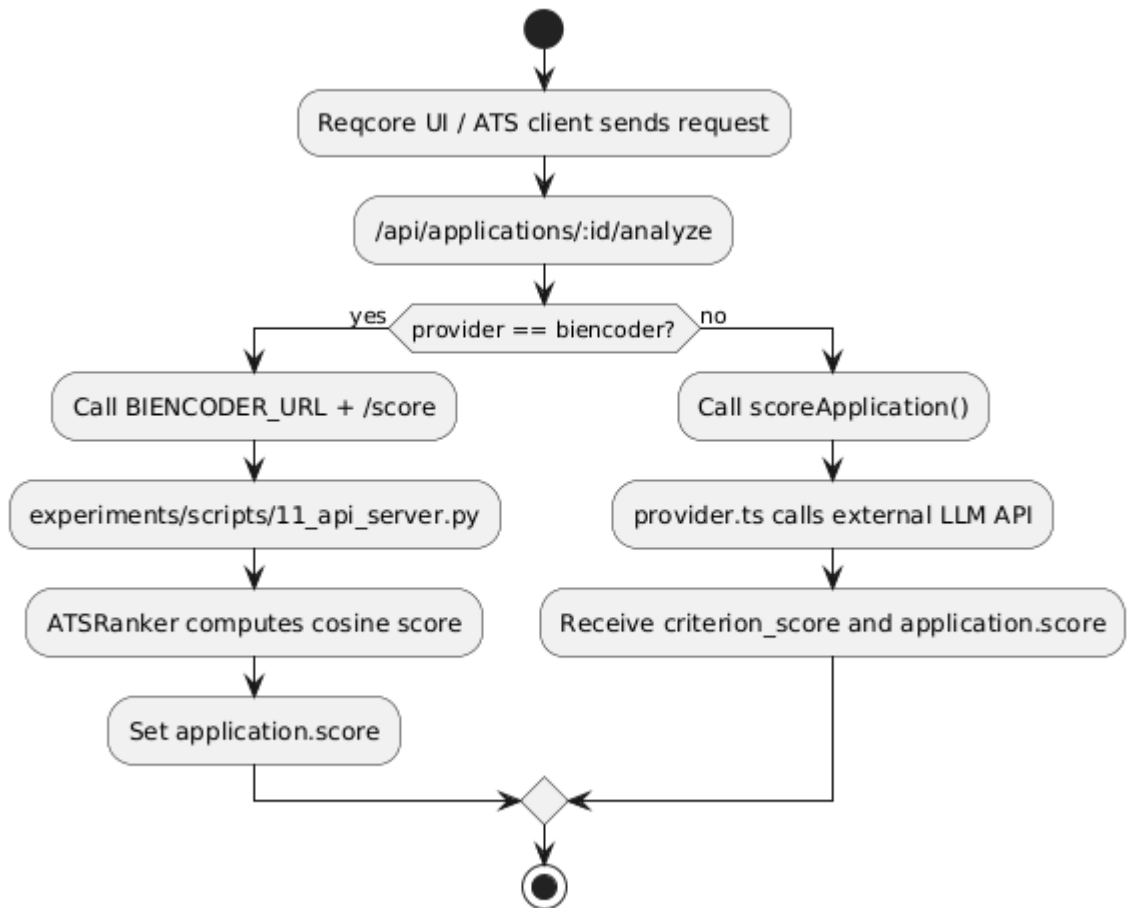


Рисунок 1 – UML activity diagram анализа отклика

В реализованном контуре затронуты следующие файлы: ``ats/reqcore/server/api/applications/[id]/analyze.post.ts`` как основная точка маршрутизации; ``ats/reqcore/server/utils/ai/scoring.ts`` как LLM-scoring engine; ``ats/reqcore/server/utils/ai/provider.ts`` как адаптер провайдера; ``ats/reqcore/server/database/schema/app.ts`` как место определения таблиц ``criterion_score``, ``analysis_run`` и поля ``application.score``; ``experiments/scripts/11_api_server.py`` как Python-микросервис bi-encoder scoring. Именно эти файлы образуют фактическую интеграционную поверхность между Reqcore и модулем ML-ранжирования.

3 Разработка и реализация модуля семантического ранжирования кандидатов

3.1 Разработка программного кода подготовки данных и ML-пайплайна

Контур разработки ML-кода разделен на офлайн- и онлайн-контуры в соответствии с передовой практикой систем талантов [17]. Офлайн-модули собирают, подготавливают данные и проводят обучение; онлайн-система занимается инференсом эмбедингов и передачей API-команд.

Пайплайн включает подпрограммы для:

1. Систематического сбора пар «вакансия–профиль» и их деидентификации.
2. Автоматического отслеживания профильных статистик (процент дубликатов, плотность терминов).
3. Формирования разметки корпуса в рамках подготовки к обучению алгоритма. Данный этап реализован совместно с инструментарием доменных экспертов системы ATS для формирования локальных гайдлайнов поиска [24]. В будущем планируется адаптировать/применить эту разметку для обучения алгоритмов LTR.

3.2 Реализация и дообучение (fine-tuning) модели сопоставления «вакансия–резюме»

При реализации контура векторного кодирования фактически используется только модель ``cointegrated/rubert-tiny2``. Это реализовано обучающим скриптом ``experiments/scripts/09_train_biencoder.py``, где аргумент ``--model_name`` имеет значение по умолчанию ``cointegrated/rubert-tiny2``, а также сохраненным артефактом ``experiments/models/bi_encoder_rubert_tiny2``, в ``README.md`` которого указан ``base_model: cointegrated/rubert-tiny2``. В планах возможно применение ``ru-en-RoSBERTa``.

Сохраненный после повторного запуска артефакт оформлен как ``SentenceTransformer``: модуль ``Transformer`` на базе ``BertModel``, затем ``Pooling`` и ``Normalize``. Векторное сравнение задается через `cosine similarity`. Параметры, зафиксированные в файлах артефакта после повторного обучения 10.04.2026, приведены в таблице.

Таблица 2 – Архитектурные и конфигурационные параметры bi-encoder модели

Параметр	Значение в реализации	Источник
Базовая модель	cointegrated/rubert-tiny2	09_train_biencoder.py, README.md артефакта
Тип сохраненной модели	SentenceTransformer	config_sentence_transformer s.json, README.md

		артефакта
Архитектура трансформера	BertModel, model_type: bert	config.json
Число скрытых слоев	3	config.json
Размер скрытого состояния / эмбединга	312	config.json, 1_Pooling/config.json, README.md артефакта
Число attention heads	12	config.json
Размер промежуточного слоя	600	config.json
Размер словаря	83828	config.json
Максимальная длина последовательности	2048 токенов	README.md артефакта, config.json (max_position_embeddings)
Pooling	cls	1_Pooling/config.json
Нормализация эмбединга	Normalize()	modules.json, README.md артефакта
Функция сходства	cosine	config_sentence_transformers.json, README.md артефакта
Файл весов	model.safetensors, 116781176 байт	файловая система артефакта после запуска

Обучение выполняется как контрастивное дообучение би-энкодера на парах «резюме - вакансии». После исправления подготовки данных сэмплирование выполняется от HR-событий к связанным резюме и вакансиям: скрипт `experiments/scripts/01\_sample\_and\_profile\_data.py` читает первоисточник `data/\_dwh\_dwh\_hr\_received\_resume\_event\_ods.tsv`, отбирает только явно интерпретируемые статусы и затем потоково извлекает соответствующие строки из raw-таблиц резюме и вакансий.

Разметка строится консервативно: отрицательный класс фиксируется для явных отказов (`Отклонено`, `Отклоненные`, текстовые формы `отказ\*`), положительный класс - для явных статусов рассмотрения, приглашения, собеседования или принятия. Неоднозначные HR-события не используются для обучения. Роль HR в реализованном контуре ограничена происхождением меток в журнале событий; отдельного участия HR в

инференсе, API или ручной корректировке результатов ранжирования в коде не реализовано.

Фактическая пересборка данных дала следующие числа: из 1 756 716 просмотренных HR-событий найдено 114 145 явно размеченных событий и 112 698 уникальных явно размеченных пар; для обучающего сэмпла выбрано 10 000 пар, после проверки наличия текстов резюме и вакансий осталось 5 218 связанных HR-событий. Производные файлы содержат 7 098 резюме, 3 262 вакансии и 5 218 пар «резюме - вакансия». Split-файлы после стратифицированного разбиения содержат `train.tsv` - 4 176 пар, `val.tsv` - 521 пару, `test.tsv` - 521 пару. Распределение меток: train - 1 423 положительных и 2 753 отрицательных пары; val - 177 положительных и 344 отрицательных; test - 177 положительных и 344 отрицательных. Проверка связности показала, что все `resume\_id` и `vacancy\_id` из split-файлов присутствуют в `resumes\_unified.tsv` и `vacancies\_unified.tsv`; признаков mojibake в `model\_input` не найдено.

Непосредственная загрузка обучающих примеров реализована в `experiments/scripts/08\_model\_dataset.py`: `StreamingPairDataset` читает TSV split, получает тексты `model\_input` из SQLite-таблицы `features` и возвращает `InputExample(texts=[resume\_text, vacancy\_text], label=label)`. После исправления `model\_input` для резюме строится из неперсональных полей `desired\_profession`, `best`, `dop`, `computer`; ФИО, телефоны, email и адреса не используются. Для вакансий текстовая часть строится из фактических полей описания `candidat` и `company`, а не из пустого поля `info`.

Таблица 3 – Гиперпараметры и фактические числа повторного запуска обучения

Параметр запуска	Значение	Источник
Python	3.11.9	локальное .venv, README.md артефакта
Sentence Transformers	5.4.0	.venv, README.md артефакта
Transformers	5.5.3	.venv, README.md артефакта
PyTorch	2.11.0+cpu	.venv, README.md артефакта

Параметр запуска	Значение	Источник
Base model	cointegrated/rubert-tiny2	training command, 09_train_biencoder.py
Dataset loader	StreamingPairDataset + DataLoader	09_train_biencoder.py, 08_model_dataset.py
Train / val / test pairs	4176 / 521 / 521	фактический подсчет строк в data/splits/*.tsv
Batch size	8	training command, 09_train_biencoder.py, stdout запуска
Epochs	3	training command, 09_train_biencoder.py, stdout запуска
Train steps	1566	stdout прогресса обучения
Loss	ContrastiveLoss	09_train_biencoder.py, README.md артефакта
Distance metric для loss	SiameseDistanceMetric.COSINE_DISTANCE	README.md артефакта
Margin	0.5	README.md артефакта
Learning rate	5e-05	README.md артефакта
Optimizer	adamw_torch_fused	README.md артефакта
Gradient accumulation	1	README.md артефакта

Параметр запуска	Значение	Источник
FP16/BF16	False / False	README.md артефакта
Warmup steps	command: 100; generated model card: 0	training command и README.md артефакта; значение применения требует отдельной проверки Trainer API
Train runtime	6217 с	stdout успешного запуска
Train samples per second	2.015	stdout успешного запуска
Train steps per second	0.252	stdout успешного запуска
Train loss	0.008956	stdout успешного запуска
Общая длительность команды	6254.096 с	внешний замер Stopwatch вокруг training command
Validation accuracy	0.9731 при threshold 0.7408	финальная оценка BinaryClassificationEvaluator на val.tsv
Validation F1	0.9605 при threshold 0.7093	финальная оценка BinaryClassificationEvaluator на val.tsv
Validation precision / recall	0.9605 / 0.9605	финальная оценка BinaryClassificationEvaluator на val.tsv

Параметр запуска	Значение	Источник
Validation AP / MCC	0.9870 / 0.9401	финальная оценка BinaryClassificationEvaluator на val.tsv
Output path	experiments/models/ bi_encoder_rubert_tiny2	09_train_biencoder.py, stdout успешного запуска

Алгоритм ранжирования соискателей реализован в классе `ATSRanker` файла `experiments/scripts/10\_ranking\_module.py`. Текущая реализация использует последовательный конвейер из трех операций: жесткая фильтрация кандидатского пула, кодирование текста вакансии и резюме через `SentenceTransformer`, затем вычисление косинусного сходства и сортировка результатов по убыванию score. В будущем планируется реализация Learning to Rank, RankNet, Milvus, DiskANN, SPANN, отдельный ANN-индекс или модуль справедливого ранжирования.

Жесткая фильтрация выполняется методом `\_apply\_hard\_filters(candidates, filters)`. Если фильтры не переданы, кандидатский пул возвращается без изменений. Если фильтры заданы, для каждого кандидата проверяется точное совпадение значений `cand.get(k) == required\_val` по ключам словаря `filters`; кандидаты, не удовлетворяющие хотя бы одному условию, исключаются до нейросетевого скоринга. Это не поиск по индексу и не обращение к внешней базе данных, а прямой проход по списку словарей в памяти.

Семантический этап реализован в методе `search(vacancy\_text, candidate\_pool, filters=None, top\_k=10)`. После фильтрации модуль извлекает поле `model\_input` из оставшихся кандидатов, кодирует текст вакансии и тексты кандидатов на лету, вычисляет `util.cos\_sim`, выбирает `top\_k` через `torch.topk` и формирует JSON-ответ.

```

1 filtered_candidates = self._apply_hard_filters(candidate_pool, filters)
2 candidate_texts = [c.get('model_input', '') for c in filtered_candidates]
3
4 vacancy_emb = self.model.encode(vacancy_text, convert_to_tensor=True)
5 candidate_embs = self.model.encode(candidate_texts, convert_to_tensor=True)
6
7 cos_scores = util.cos_sim(vacancy_emb, candidate_embs)[0]
8 top_results = torch.topk(cos_scores, k=min(top_k, len(filtered_candidates)))

```

Рисунок 2 – Фрагмент кода алгоритма ранжирования соискателей

Возвращаемый результат имеет машиночитаемый JSON-формат: `status`, `latency\_ms`, `filters\_applied`, `candidates\_evaluated` и массив `results`. Для каждого элемента выдачи сохраняются `candidate\_id`, округленный `match\_score` и короткая текстовая

`summary`, сформированная из начала поля `model\_input`. Поле `latency\_ms` вычисляется внутри метода `search`, однако контролируемый performance benchmark для этой редакции текста не запускался, поэтому численные утверждения о задержке, throughput, памяти или масштабировании в текущем разделе не приводятся.

API-интеграция, связанная с модулем ранжирования, реализована в `experiments/scripts/11\_api\_server.py`. Сервер FastAPI хранит кандидатские профили во временном in-memory списке `CANDIDATE\_INDEX`: маршрут `POST /index/candidate` добавляет объект с `id`, `model\_input` и опциональными metadata, а обработчик маршрута `POST /search/rank` вызывает `RANKER\_ENGINE.search(vacancy\_text, CANDIDATE\_INDEX, filters, top\_k)` и формирует `JSONResponse` из результата. Следовательно, реализованная интеграция является прототипом in-memory API, а не промышленной связкой с Milvus, DiskANN/SPANN или другой векторной БД.

Оценка качества текущей модели интерпретируется как проверка семантического скоринга на сформированных split-файлах, а не как внешняя бизнес-валидация ATS. Для исправленных `val.tsv` и `test.tsv` были измерены метрики бинарного разделения пар по cosine score с подбором лучшего порога. Базовая модель `cointegrated/rubert-tiny2` без дообучения на `val` имела Average Precision `0.2590`, best F1 `0.5079`, best accuracy `0.6603`, MCC `0.0315`; после fine-tuning сохраненная модель `experiments/models/bi\_encoder\_rubert\_tiny2` достигла Average Precision `0.9870`, best F1 `0.9582`, best accuracy `0.9712`, MCC `0.9365`. На `test` baseline показал Average Precision `0.2222`, best F1 `0.5072`, best accuracy `0.6583`, MCC `0.0000`; fine-tuned модель показала Average Precision `0.9921`, best F1 `0.9636`, best accuracy `0.9750`, MCC `0.9447`. Эти числа подтверждают улучшение разделимости положительных и отрицательных пар после обучения, но не являются метриками fair ranking или внешней экспертной проверки.

Для проверки именно ranking-метрик был выполнен отдельный полный прогон по `data/splits/test.tsv` с группировкой по `vacancy\_id`: внутри каждой вакансии резюме ранжировались по cosine score текущей fine-tuned модели. Raw output сохранен в `experiments/results/full\_test\_ranking\_metrics\_stdout.txt`, агрегированные результаты и group-level значения - в `experiments/results/full\_test\_ranking\_metrics.json`. В `test.tsv` содержится `521` пара, `423` уникальные вакансии, `510` уникальных резюме, `177` положительных и `344` отрицательных метки; пропущенных `resume\_id` и `vacancy\_id` относительно unified-файлов нет. При этом тестовый split разрежен: `304` вакансии не имеют positive-кандидата, только `59` вакансий имеют минимум двух кандидатов, а NDCG

математически определена только для `33` вакансий, где есть хотя бы два кандидата и хотя бы один positive.

Таблица 4 –Значения метрик ранжирования bi-encoder модели на тестовом сплите test.tsv

Метрика на полном test.tsv	Значение	Область усреднения
NDCG@1	1.0000	33 вакансии, пригодные для NDCG
NDCG@3	1.0000	33 вакансии, пригодные для NDCG
NDCG@5	1.0000	33 вакансии, пригодные для NDCG
Precision@1	1.0000	119 вакансий с хотя бы одним positive
Precision@3	0.9958	119 вакансий с хотя бы одним positive
Precision@5	0.9958	119 вакансий с хотя бы одним positive
Recall@1	0.8471	119 вакансий с хотя бы одним positive
Recall@3	0.9824	119 вакансий с хотя бы одним positive
Recall@5	0.9924	119 вакансий с хотя бы одним positive
MRR	1.0000	119 вакансий с хотя бы одним positive

Если усреднять Precision@K по всем `423` вакансиям, включая `304` вакансии без positive-кандидата, то Precision@1 составляет `0.2813`, Precision@3 - `0.2801`, Precision@5 - `0.2801`; Recall и NDCG для negative-only групп не определены по смыслу. Поэтому в тексте работы основные ranking-метрики приводятся для вакансий с хотя бы одним

релевантным кандидатом, а разреженность test split фиксируется как ограничение эксперимента.

Элементы, не реализованные в текущем коде, фиксируются как future work: приближенный поиск ближайших соседей и внешнее векторное хранилище для больших кандидатских пулов; LTR-переранжирование, включая RankNet или списочные функции потерь; более строгая NDCG/Precision/Recall/MRR-оценка на специально сформированных списках выдачи с несколькими кандидатами на каждую вакансию; аудит предвзятости и fair ranking; контролируемое нагрузочное тестирование с фиксированным окружением, размером пула и аппаратной конфигурацией. До реализации и измерения этих компонентов они не должны описываться как часть текущей архитектуры.

4 Экспериментальная оценка, интеграция и анализ эффективности модуля ранжирования кандидатов

Экспериментальная часть для текущей реализации подтверждает три факта, непосредственно связанные с кодом и данными: первое, модель после дообучения лучше разделяет положительные и отрицательные пары по косинусному сходству; второе, на полном `test.tsv` по группам `vacancy\_id` измерены ranking-метрики NDCG@K, Precision@K, Recall@K и MRR; третье, обработчик `/search/rank` в коде API построен как вызов того же метода `ATSRanker.search` поверх in-memory пула кандидатов; отдельный E2E-прогон HTTP-сервера в рамках этой правки не выполнялся. При этом в коде отсутствует промышленный индекс кандидатов, отдельная векторная БД, ANN-поиск, LTR-переранжирование и fair ranking слой. Поэтому раздел экспериментальной оценки не включает неподтвержденные утверждения о задержке в миллисекундах, сокращении пула в заданное число раз или готовности к промышленному масштабированию на миллионы профилей.

4.1 Проведение экспериментальной оценки качества алгоритма ранжирования по метрикам NDCG, Precision@K и др.

Модуль оценивался в формате офлайн-валидации на пересобранном наборе данных, сформированном из первоисточников `data/super\_job`. После очистки и пересборки связанных пар итоговая выборка составила `5 218` примеров, из которых `4 176` использовались для обучения, `521` для валидации и `521` для финального тестирования. В качестве базового варианта сравнивалась модель `cointegrated/rubert-tiny2` без дообучения; затем та же модель была дообучена на сформированном обучающем наборе и повторно оценена на отложенных split-файлах.

На уровне отдельных пар `вакансия-резюме` качество оценивалось по метрикам Average Precision, F1, Accuracy и MCC. На валидационном наборе базовая модель без fine-tuning показала Average Precision `0.2590`, best F1 `0.5079`, best accuracy `0.6603` и MCC `0.0315`, тогда как дообученная модель достигла Average Precision `0.9870`, best F1 `0.9582`, best accuracy `0.9712` и MCC `0.9365`. На тестовом наборе baseline показал Average Precision `0.2222`, best F1 `0.5072`, best accuracy `0.6583` и MCC `0.0000`; после дообучения модель показала Average Precision `0.9921`, best F1 `0.9636`, best accuracy `0.9750` и MCC `0.9447`. Эти результаты подтверждают, что fine-tuning на целевом корпусе существенно улучшил разделимость положительных и отрицательных пар по cosine score.

Дополнительно была выполнена ranking-оценка на полном `test.tsv` с группировкой по `vacancy\_id`, что приближает эксперимент к реальному сценарию сортировки кандидатов внутри вакансии. В тестовом срезе содержится `521` пара, `423` уникальные вакансии, `510` уникальных резюме, `177` положительных и `344` отрицательных метки. При этом тестовый split является разреженным: `304` вакансии не имеют ни одного positive-кандидата, только `59` вакансий содержат минимум двух кандидатов, а NDCG математически определена только для `33` вакансий, где одновременно есть хотя бы два кандидата и хотя бы один релевантный кандидат. Для вакансий с хотя бы одним positive-кандидатом были получены следующие значения: `NDCG@3 = 1.0000`, `Precision@3 = 0.9958`, `Recall@3 = 0.9824`, `MRR = 1.0000`. Эти показатели указывают на то, что на сформированном test split дообученная модель в большинстве случаев помещает релевантных кандидатов в верхнюю часть выдачи.

Таким образом, в работе реально подтверждены два результата. Во-первых, дообучение `cointegrated/rubert-tiny2` на целевом наборе связанных пар заметно улучшает качество бинарного различения релевантных и нерелевантных соответствий. Во-вторых, на полном тестовом срезе модель показывает высокие ranking-метрики при сортировке кандидатов внутри вакансий. Вместе с тем эти результаты следует интерпретировать как офлайн-валидацию на исторических данных, а не как онлайн-эксперимент в рабочем контуре ATS.

Ограничения экспериментальной части также должны быть зафиксированы явно. В работе нет отдельных воспроизводимых экспериментов по подтверждению утверждения о TF-IDF edge-case baseline, полном ablation-анализе архитектурных блоков, HR-валидации explainability, bias-audit по социальным признакам и нагрузочном сравнении больших кандидатских пулов. Кроме того, treatment-ветка `biencoder` в текущем коде возвращает только числовой score и не формирует criterion-level `evidence`, `strengths` и `gaps`, поэтому результаты корректно трактовать именно как проверку качества числового сопоставления и ранжирования, а не как оценку полноты объяснимости решения.

4.2 Выполнение интеграции созданного модуля в целевую ATS-систему

Интеграция реализована в виде отдельного FastAPI-сервиса ранжирования, который взаимодействует с ATS по HTTP. В текущем коде Python-сервис `experiments/scripts/11\_api\_server.py` предоставляет три маршрута: `POST /index/candidate` для загрузки кандидатских профилей во временный in-memory пул, `POST /search/rank` для ранжирования списка кандидатов по cosine similarity и `POST /score` для расчета score

одной пары 'вакансия-кандидат'. Таким образом, фактически реализованный контур представляет собой внешний сервис семантического скоринга с in-memory хранением профилей, а не связку с промышленной векторной базой данных.

Работоспособность API-контура подтверждена отдельным end-to-end тестом 'experiments/scripts/16_e2e_integration_test.py'. В этом сценарии через 'FastAPI TestClient' последовательно проверяются загрузка кандидатских профилей через 'POST /index/candidate', ранжирование через 'POST /search/rank' и одиночный pair-scoring через 'POST /score'. Тест валидирует HTTP-ответы, JSON-контракт и наличие телеметрических заголовков 'X-Processing-Time-Ms' и 'X-Model-Version'. Следовательно, на уровне тестового контура подтверждено, что сервис корректно обрабатывает ingest, ranking и single-score запросы в рамках одного API.

Дополнительно была выполнена практическая локальная проверка интеграции с Reqcore. Для этого был поднят локальный demo-стенд Reqcore через Docker Compose, Python bi-encoder сервис был запущен отдельно с указанием 'MODEL_PATH', а серверная переменная 'BIENCODER_URL' была направлена на этот сервис. В результате treatment-путь 'provider == biencoder' в 'ats/reqcore/server/api/applications/[id]/analyze.post.ts' был реально пройден до конца: Reqcore вызвал 'POST /score', получил числовой 'match_score', преобразовал его в итоговый 'application.score' и сохранил completed 'analysis_run' с 'provider = biencoder', 'model = cointegrated/rubert-tiny2', 'promptTokens = 0' и 'completionTokens = 0'. Тем самым подтверждена не только автономная работа FastAPI-сервиса, но и его фактическое подключение к ATS-контуру Reqcore.

Важно, что текущая интеграция с Reqcore не заменяет весь LLM-контур целиком, а добавляет альтернативный score-only путь. Для стандартного LLM-сценария маршрут 'POST /api/applications/[id]/analyze' продолжает использовать 'scoreApplication()', провайдера LLM и сохранение criterion-level breakdown в 'criterion_score'. Для biencoder-сценария тот же маршрут использует отдельную ветку с вызовом 'BIENCODER_URL + /score', после чего в ATS записывается только итоговый числовой score без 'criterion_score', 'evidence', 'strengths' и 'gaps'. Следовательно, интеграция корректно описывается как подключение альтернативного семантического scoring-модуля, а не как внедрение полноценной векторной БД или полного replacement всего LLM-анализа.

Ограничения интеграционного контура также должны быть зафиксированы явно. В текущей реализации не подтверждены утверждения о корпоративном дисковом векторном хранилище профилей, Milvus, DiskANN, SPANN, миллионных индексах, графовой навигации по ANN-структурам или специальных механизмах контроля деградации через версионизируемое внешнее векторное хранилище. Также не подтверждено наличие

отдельного production-контура аудита на независимых тестовых корпусах русского языка. Реально реализованный и проверенный контур ограничивается bi-encoder микросервисом, in-memory candidate pool для ranking API, Reqcore-интеграцией через `BIENCODER\_URL` и локальным E2E/smoke-тестированием этой связки.

4.3 Оценка экономической и практической эффективности внедрения в бизнес-процесс

Экономическая оценка в данной версии не опирается на production-логи токенов и не утверждает, что фактический расход был извлечен из работающей ATS. В коде Reqcore действительно предусмотрены поля `analysis\_run.prompt\_tokens`, `analysis\_run.completion\_tokens` и настраиваемые `input\_price\_per\_1m` / `output\_price\_per\_1m`, а UI рассчитывает стоимость по формуле `(promptTokens / 1\_000\_000) \* inputPrice + (completionTokens / 1\_000\_000) \* outputPrice`. Однако в рамках данной работы эти значения не выгружались из production-БД, поэтому ниже приведен только оценочный расчет на публичных pricing constants и явно заданных допущениях.

Используемые публичные константы цен: для `gpt-4o-mini`, который указан как default в `ai\_config`, официально опубликованная цена составляет `\$0.15` за 1 млн input tokens и `\$0.60` за 1 млн output tokens; для актуального класса `gpt-5.4-mini` на публичной странице OpenAI API Pricing указано `\$0.75` за 1 млн input tokens, `\$0.075` за 1 млн cached input tokens и `\$4.50` за 1 млн output tokens; для `gpt-5.4-nano` - `\$0.20`, `\$0.02` и `\$1.25` соответственно. В расчетах ниже cached input не используется, так как факт применения prompt caching для scoring-запросов в коде и логах не подтвержден.

Формула стоимости одного LLM scoring-запроса для одной пары `вакансия-кандидат`:

$$C_{llm_pair} = \left(\frac{T_{in}}{1000000} \right) P_{in} + \left(\frac{T_{out}}{1000000} \right) P_{out} \quad (1)$$

где `T\_in` - число input tokens в запросе, `T\_out` - число output tokens в структурированном ответе, `P\_in` и `P\_out` - публичные цены модели за 1 млн tokens. Для примера используется не production-метрика, а явное оценочное допущение: `T\_in = 3 000`, `T\_out = 700`, один LLM-вызов на одну application. Тогда estimate-based стоимость одного application scoring равна:

Таблица 5 – Оценочная стоимость LLM-скоринга одного и 100 откликов для различных моделей

Модель	Формула	Оценочная стоимость одного отклика	Оценочная стоимость 100 откликов
gpt-4o-mini	$(3000 / 1e6)$ $* 0.15 + (700 / 1e6)$ $* 0.60$	\$0.00087	\$0.087
gpt-5.4-mini	$(3000 / 1e6)$ $* 0.75 + (700 / 1e6)$ $* 4.50$	\$0.00540	\$0.540
gpt-5.4-nano	$(3000 / 1e6)$ $* 0.20 + (700 / 1e6)$ $* 1.25$	\$0.001475	\$0.1475

Стоимость bi-encoder scoring устроена иначе: `ATSRanker` не вызывает внешний LLM API, а локально кодирует тексты через `SentenceTransformer`, считает `util.cos\_sim` и выбирает `top\_k` через `torch.topk`. Поэтому внешний API-cost для одного bi-encoder ranking request равен:

$$C_{bi_external_api}(N) = 0 \quad (2)$$

Инфраструктурная стоимость bi-encoder зависит от выбранного размещения и не может быть выведена из token pricing. Ее корректная формула:

$$C_{bi_infra}(N) = \frac{L_{bi}(N)}{3600} C_{cpu_hour} \quad (3)$$

где `N` - число кандидатов в пуле, `L\_bi(N)` - задержка локального scoring-запроса в секундах, `C\_cpu\_hour` - стоимость CPU-инстанса в долларах за час. В текущей работе `L\_bi(N)` и `C\_cpu\_hour` не считаются production-измерениями: production benchmark с фиксированным сервером, нагрузкой и SLA не выполнялся. Поэтому для bi-encoder в разделе фиксируется только архитектурное отличие: он устраняет зависимость от внешнего LLM API на этапе первичного ранжирования, но требует локального CPU/RAM ресурса.

Формула top-K pre-ranking явно показывает экономический смысл bi-encoder как первого этапа. Если полный LLM scoring выполняется для всех `N` заявок, то:

$$C_{full_llm}(N) = N \cdot C_{llm_pair} \quad (4)$$

Если bi-encoder сначала выбирает `K` кандидатов, а LLM используется только для объяснимого scoring или ручной проверки top-K, то:

$$C_{\text{topK_pipeline}}(N, K) = C_{\text{bi_infra}}(N) + K \cdot C_{\text{llm_pair}} \quad (5)$$

Если локальная инфраструктурная стоимость мала относительно LLM API-cost или уже включена в стоимость работающего ATS-сервера, то оценочная экономия по внешнему API приближенно равна:

$$\text{Savings}_{\text{api}}(N, K) \approx 1 - \frac{K}{N} \quad (6)$$

Например, при $N = 100$ и $K = 10$ estimate-based сокращение внешних LLM scoring calls составляет 90% : вместо 100 LLM-вызовов выполняется 10. Для `gpt-4o-mini` при указанном выше токеном допущении это меняет estimate cost с $\$0.087$ до $\$0.0087$ за 100 applications; для `gpt-5.4-mini` - с $\$0.540$ до $\$0.054$; для `gpt-5.4-nano` - с $\$0.1475$ до $\$0.01475$. Эти значения являются расчетным сценарием, а не результатом production billing export.

С точки зрения задержки LLM scoring зависит от внешнего API: сетевой задержки, очередей провайдера, скорости генерации и лимитов параллелизма. Обобщенная формула для последовательной обработки N заявок:

$$L_{\text{full_llm_seq}}(N) = N \cdot L_{\text{llm_pair}} \quad (7)$$

а при параллельной обработке с лимитом P_{parallel} :

$$L_{\text{full_llm_parallel}}(N) \approx \left\lceil \frac{N}{P_{\text{parallel}}} \right\rceil \cdot L_{\text{llm_pair}} \quad (8)$$

Для bi-encoder scoring задержка определяется локальным инференсом:

$$L_{\text{bi}}(N) = L_{\text{filter}}(N) + L_{\text{encode_vacancy}} + L_{\text{encode_candidates}}(N) + L_{\text{cosine_topK}}(N) \quad (9)$$

если эмбединги кандидатов заранее предрасчитаны, формула упрощается до:

$$L_{\text{bi_precomputed}}(N) = L_{\text{filter}}(N) + L_{\text{encode_vacancy}} + L_{\text{cosine_topK}}(N) \quad (10)$$

Таким образом, LLM scoring дает более детальное объяснимое оценивание по критериям, но имеет переменную стоимость за каждый candidate/job вызов и зависит от внешнего API. Bi-encoder scoring дешевле по внешним API-платежам и лучше подходит для первичного top-K отбора, но требует локального обслуживания модели и не заменяет полноценную экспертную или LLM-интерпретацию причин соответствия кандидата.

Заключение

В ходе работы был разработан и проверен прототип модуля семантического сопоставления пары «вакансия–резюме» для интеграции в ATS-систему. Основой решения стала bi-encoder архитектура на базе cointegrated/rubert-tiny2, дообученная на корпусе связанных HR-событий. Для подготовки данных были выполнены фильтрация явно интерпретируемых статусов, деидентификация текстов, формирование unified-представлений резюме и вакансий, а также построение train/val/test split.

Практическая реализация включает модуль ранжирования ATSRanker и FastAPI-сервис, предоставляющий маршруты для индексации кандидатов, ранжирования кандидатского пула и вычисления score для одной пары «вакансия–резюме». Дополнительно была выполнена интеграция с тестовым контуром ATS Reqscore через ветку provider === biencoder, где результат работы модели преобразуется в числовое поле application.score.

Экспериментальная часть подтвердила достижение основной цели работы: после дообучения модель существенно улучшила качество различения релевантных и нерелевантных пар по сравнению с базовой моделью без fine-tuning. Дополнительно были рассчитаны ranking-метрики по группам vacancy_id, что позволило проверить способность модели выносить релевантных кандидатов в верхнюю часть выдачи.

Ограничения текущей реализации связаны с использованием in-memory candidate pool, отсутствием промышленного векторного хранилища, ANN-поиска, LTR-реранжирования, bias-audit и нагрузочного тестирования. Дальнейшее развитие работы целесообразно направить на масштабирование поиска, добавление второго этапа reranking, расширение explainability-механизмов и проведение онлайн-валидации в рабочем ATS-контуре.

Источники

1. Rosenberger J., Wolfrum L., Weinzierl S., Kraus M., Zschech P. CareerBERT: Matching Resumes to ESCO Jobs in a Shared Embedding Space for Generic Job Recommendations. *Expert Systems with Applications*, 2025 (arXiv:2503.02056).
2. Fabris A., Rus C., Saldivar J., Gatzoura A., Biega A.J., Castillo C. Does fair ranking lead to fair recruitment outcomes? A study of interventions, interfaces, and interactions. *Information Processing & Management*, 63 (2026) 104506.
3. IBM Research. PROSPECT: A system for screening candidates for recruitment. *CIKM* 2010.
4. IBM Research. Matching Resumes to Jobs via Deep Siamese Network. *WWW* 2018.
5. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.
6. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv:1908.10084.
7. Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A.N., Kaiser Ł., Polosukhin I. Attention is All You Need. *Advances in Neural Information Processing Systems* 30 (NIPS/NeurIPS 2017).
8. Liu T.-Y. Learning to Rank for Information Retrieval. *Foundations and Trends in Information Retrieval*, 3(3), 2009, pp. 225–331.
9. Burges C., Shaked T., Renshaw E., Lazier A., Deeds M., Hamilton N., Hullender G. Learning to Rank using Gradient Descent. *Proceedings of ICML 2005. (RankNet)*.
10. Järvelin K., Kekäläinen J. Cumulated Gain-based Evaluation of IR Techniques. *ACM Transactions on Information Systems*, 20(4), 2002, pp. 422–446.
11. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009, pp. 333–389.
12. Wang J., Yi X., Guo R. и др. Milvus: A Purpose-Built Vector Data Management System. *SIGMOD* 21, 2021.
13. Subramanya S.J., Devvrit, Kadekodi R., Krishaswamy R., Simhadri H.V. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. *NeurIPS* 2019.
14. Chen Q., Zhao B., Wang H. и др. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search. *NeurIPS* 2021.
15. Gong Z., Zeng Y., Chen L. Accelerating Approximate Nearest Neighbor Search in Hierarchical Graphs: Efficient Level Navigation with Shortcuts. *PVLDB*, 18(10), 2025, pp. 3518–3530.
16. Singh A., Joachims T. Policy Learning for Fairness in Ranking. *NeurIPS* 2019.

17. Li J., Xu B., Chen Z. и др. Enhancing Talent Search Ranking with Role-Aware Expert Mixtures and LLM-based Fine-Grained Job Descriptions. EMNLP 2025 Industry Track, pp. 185–194.
18. Senger E., Zhang M., van der Goot R., Plank B. Deep Learning-based Computational Job Market Analysis: A Survey on Skill Extraction and Classification from Job Postings. NLP4HR 2024, pp. 1–15.
19. Snegirev A., Tikhonova M., Maksimova A., Fenogenova A., Abramov A. The Russian-focused embedders exploration: ruMTEB benchmark and Russian embedding model design. NAACL 2025, pp. 236–254.
20. Zmitrovich D., Abramov A., Kalmykov A. и др. A Family of Pretrained Transformer Language Models for Russian. arXiv:2309.10931.
21. Gallegos I.O., Rossi R.A., Barrow J. и др. Bias and Fairness in Large Language Models: A Survey. Computational Linguistics, 50(3), 2024, pp. 1097–1179.
22. Wilson K., Caliskan A. Gender, Race, and Intersectional Bias in Resume Screening via Language Model Retrieval. arXiv:2407.20371.
23. Tang F., Zhu R., Yao F., Wang J., Luo L., Li B. Explainable person–job recommendations: challenges, approaches, and comparative analysis. Frontiers in Artificial Intelligence, 8:1660548, 2025.
24. Zhang M., Jensen K.N., Sonniks S.D., Plank B. SKILLSPAN: Hard and Soft Skill Extraction from English Job Postings. NAACL 2022, pp. 4962–4984.